

北京理工大学

机器学习初步实验报告

作业 3: Q-learning 和 DQN

学 院:	计算机学院
专 业:	计算机科学与技术
班 级:	07112304
学生姓名:	陈墨霏
学 号:	1120233329

2025 年 5 月 13 日

第一部分 Q-Learning 算法的实现和测试

任务 1: Q-Learning 算法补全与描述

补全 RL.py 中 Q-learning 方法的代码，提交代码实现截图，并解释实现思路。

实现思路

首先对算法所用到的函数和变量进行初始化。这包括：对 Q 函数的初始化；初始化动作数量 `nActions`、奖励列表 `rewardList`、温度参数 `temperature`。特别要说明的是，这里为温度参数设置最小值为 1，是为了防止温度过小导致在 Boltzmann exploration 的计算时溢出。

之后，进入训练阶段。训练阶段是一个次数为 `nEpisodes` 的大循环，取回合数 `episode` 作为计数变量，表明每一次循环是一个回合（episode），可以形象理解为从一局游戏开始到游戏结束。每一次循环，先初始化当前状态为 `s0`，之后进入一个小循环，取步数 `step` 作为计数变量，表明每一次循环是一个步（step），可以形象理解为在当前局游戏中进行了一次操作。

对于每一个 `step`，首先结合 ϵ -贪婪和 Boltzmann exploration 选择动作（action）。

ϵ -贪婪策略：在每一步中，以概率 ϵ 选择一个随机动作（探索），以概率 $1-\epsilon$ 选择当前 Q 值最大的动作（利用）。这种策略在训练初期可以更多地探索环境，而在训练后期逐渐趋向于利用已有的知识，从而在探索与利用之间取得平衡。

Boltzmann exploration：在每一步中，根据当前状态的 Q 值计算每个动作的概率分布，然后根据这个分布选择一个动作。由于分布的形式与物理上的 Boltzmann 分布类似，因此称为 Boltzmann exploration，相应的参数也就被称为 `temperature`，起到的效果也与温度在 Boltzmann 分布中起到的效果类似（`temperature` 越高，分布越接近均匀分布；`temperature` 越低，动作选择更集中于高 Q 值的动作）。这个方法可以在一定程度上避免陷入局部最优解。分布如下：

$$P(a|s) = \frac{\exp\left(\frac{Q(s,a)}{\text{temperature}}\right)}{\sum_{a'} \exp\left(\frac{Q(s,a')}{\text{temperature}}\right)}$$

选择完毕动作后，执行动作并观察环境的反馈（奖励 `reward` 和下一个状态 `nextState`）。根据奖励更新累计奖励列表 `rewardList`。根据反馈更新 Q 值。Q 值的更新公式如下：

$$Q(s, a) = Q(s, a) + \alpha * \left(\text{reward} + \gamma * \max_{a'} Q(s', a') - Q(s, a) \right)$$

其中，`alpha` 是学习率，取 $\alpha = \frac{1}{\text{episode} + 1}$ 。`gamma` 是折扣因子。一个小循环的最后，更新当前状态 `currentState` 为下一个状态 `nextState`。进行完所有的步数后，进入下一个回合。

进行完所有的回合后，训练结束。计算最终策略 `policy`。

$$\text{policy} = \underset{a}{\operatorname{argmax}} Q(s, a)$$

下面是全部的 `qLearning` 函数的代码实现截图。

```

1 def qLearning(self, s0, initialQ, nEpisodes, nSteps, epsilon=0, temperature=0):
2     """
3     qLearning算法，需要将Epsilon exploration和 Boltzmann exploration 相结合。
4     以epsilon的概率随机取一个动作，否则采用 Boltzmann exploration取动作。
5     当epsilon和temperature都为0时，将不进行探索。
6
7     Inputs:
8     s0 -- 初始状态
9     initialQ -- 初始化Q函数 (|A|x|S| array)
10    nEpisodes -- 回合 (episodes) 的数量 (one episode consists of a trajectory of nSteps that starts in s0)
11    nSteps -- 每个回合的步数(steps)
12    epsilon -- 随机选取一个动作的概率
13    temperature -- 调节 Boltzmann exploration 的参数
14
15    Outputs:
16    Q -- 最终的 Q函数 (|A|x|S| array)
17    policy -- 最终的策略
18    rewardList -- 每个episode的累计奖励 (|nEpisodes| array)
19    """
20
21    Q = np.copy(initialQ)
22    nActions = Q.shape[0]
23    rewardList = np.zeros(nEpisodes)
24    temperature = max(temperature, 1) # 防止温度过小导致溢出
25
26    for episode in range(nEpisodes):
27        currentState = s0
28        for step in range(nSteps):
29            # 结合ε-贪婪和Boltzmann exploration选择动作
30            if np.random.rand(1) < epsilon:
31                action = np.random.randint(0, nActions) # 随机选择动作
32            else:
33                # Boltzmann exploration
34                # np.exp(Q[:, currentState] / temperature)的形状是(|A|,)
35                # np.sum(np.exp(Q[:, currentState] / temperature))的形状是标量
36                # 二者相除，得到一个概率分布
37                # 其实是借助了softmax函数来进行动作选择
38                expValues = np.exp(Q[:, currentState] / temperature)
39                actionProbabilities = expValues / np.sum(expValues)
40                action = np.random.choice(nActions, p=actionProbabilities)
41
42            # 采样奖励和下一个状态
43            reward, nextState = self.sampleRewardAndNextState(currentState, action)
44
45            # 更新累计奖励
46            rewardList[episode] += reward
47
48            # Q函数更新
49            alpha = 1 / (episode + 1) # 学习率
50            Q[action, currentState] += (alpha * (reward + self.mdp.discount * np.max(Q[:, nextState]) - Q[action, currentState]))
51
52            # 更新当前状态
53            currentState = nextState
54
55    # 计算最终策略
56    policy = np.argmax(Q, axis=0)
57
58    return [Q, policy, rewardList]

```

Figure 1: Q-learning 算法补全

任务 2： 利用 TestRL.py 对 Q-learning 算法进行简单测试

运行 TestRL.py 并提交运行截图。

运行结果

运行结果如下。

```
~\Desktop\学习资料\A-GitHub\大二下\机器学习初步\lab\lab3\code > python TestRL.py
Q-learning results
[[31.68912961 34.747438 41.45967173 44.6604036 ]
 [28.54078654 38.66805463 43.94119872 54.14334448]]
[0 1 1 1]
```

Figure 2: TestRL.py 运行结果

任务 3: 利用 TestMaze.py 对 Q-learning 算法进行较为复杂的测试

运行 TestMaze.py 并提交运行截图，并分析生成的 Q-learning.jpg 的含义和曲线的走势。

运行结果和分析

运行结果如下。由于终端输出很长，这里只展示一部分。

```
~\Desktop\学习资料\A-GitHub\大二下\机器学习初步\lab\lab3\code > python TestMaze.py
epsilon=0.05
rewardList_avg [-131.50486884 -175.00181433 -49.15172359 -3.26787008 25.21727276
31.15891244 36.09179742 45.07718735 54.63163429 53.83649422
58.21766202 54.5772074 63.54986797 62.5900425 65.48240847
60.87424688 57.92013244 59.95927554 67.10665057 71.93685857
73.0373602 73.29036881 70.41637019 65.73960107 72.18988741
67.82797017 70.33459983 72.31069078 75.39026017 74.37718705
71.7402736 73.94819165 75.34563906 71.05699861 69.83672804
73.72685939 71.45062516 76.7658434 70.82347145 71.36935134
69.41409711 74.45697177 68.95332206 81.65785466 61.72658985
76.16135286 80.51377768 70.84650201 78.11444019 77.00664022
72.62729741 71.82600857 69.84076443 76.34710746 72.65136885
77.74634251 81.73816675 78.16506364 75.23343029 79.91037377
80.42608818 76.73545361 78.29045081 74.20783773 76.91314207
74.85052739 73.05771334 76.54509682 81.54394573 78.84463735
75.54378585 81.24698964 74.83533134 75.79117346 80.45367333
82.12555848 82.41831518 74.57553559 75.83653158 77.69165787
82.67048391 79.41803868 80.35517881 82.89449964 77.48183845
70.15446444 78.73789329 78.65158667 82.93466446 82.19679177
77.19874422 79.58805221 76.92622531 76.37027136 74.64422748
83.62634072 80.65600830 79.9547253 76.48312225 80.40362127]
```

Figure 3: TestMaze.py 运行结果

下面是输出的 Q-learning.jpg 的图像。

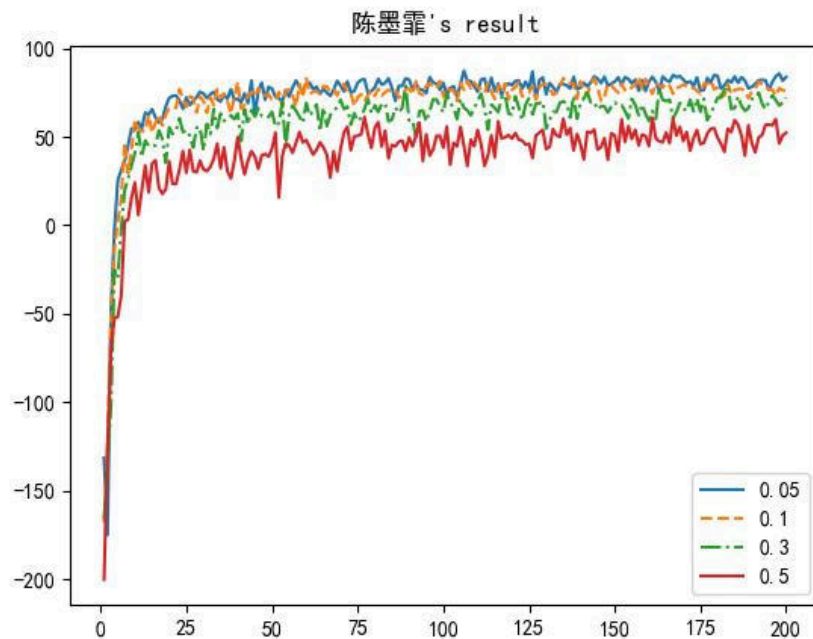


Figure 4: TestMaze.py 运行结果

理解 `TestMaze.py` 的代码逻辑后，可对该图做以下分析。

该图是一个折线图，展现的是随着 `epsilon` 值的变化，奖励 `reward` 的变化。横坐标是回合数 `episode`，纵坐标是多次（100 次）采样取平均得到的样本平均奖励列表 `rewardList_avg`。

值得注意的是，在 MDP 的奖励模型的构建当中，也有一个平均奖励（`average_reward`）的概念，往往指的是针对不同状态的间也即步数来求平均。不过这里显然指的并不是这个概念。

从图中可以看出，四条曲线虽然均不断波动，但均呈现出先增加后趋于稳定的趋势。这表明随着 `episode` 的增加，算法逐渐收敛到一个较优的策略，且策略的表现逐渐稳定。

此外，不同 `epsilon` 曲线的收敛速度和最终的稳定值也有所不同。其中，`epsilon` 为 0.05 的和为 0.1 的基本曲线重合，而之后随着 `epsilon` 增大到 0.3、0.5，曲线高度则逐步降低，奖励值减少。这表明合理的超参数设置可以改善最终策略的性能。

第二部分 利用 Q-learning 算法完成迷宫导航任务

任务 1: Q-Learning 算法补全与描述

本部分提供 `q-learning.py`，迷宫导航任务环境的相关设置已经在代码中给出，同学们需要补全 Q-learning 算法，提交补全代码部分截图。

注意该任务是随机初始化四个障碍和目的地，如果出现路径不存在的情况，请重新运行。

实现思路

这里的 Q-learning 算法与上面所描述的 Q-learning 算法基本一致，下面仅介绍作业要求补全的部分的实现思路。

动作选取部分，利用已经实现好的 `select_action` 函数进行动作选择。

```
global current_pos, epsilon_delta, epsilon
#此处需要进行动作选取,给action赋值
action = select_action(current_pos)
```

Q-learning 算法实现部分，对于未到达最终状态的情况，采用贝尔曼方程更新 Q 函数。对于到达最终状态的情况，直接用当前奖励更新 Q 函数。

```
if new_pos not in terminals:
    #此处实现Q-learning算法
    # 未到达最终状态，则采用贝尔曼方程更新Q函数
    Q[current_pos][action] += alpha * (
        current_reward + gamma * max(Q[new_pos]) - Q[current_pos][action]
    )
else:
    # 此处实现Q-learning算法
    # 到达最终状态，则直接用当前奖励更新Q函数
    Q[current_pos][action] += alpha * (current_reward - Q[current_pos][action])
```

下面是全部的 `step` 函数的代码实现截图。

```
1 def step():
2
3     global current_pos, epsilon_delta, epsilon
4     #此处需要进行动作选取,给action赋值
5     action = select_action(current_pos)
6
7     new_pos = list(current_pos)
8     if action==0 and new_pos[0]>0: #move up
9         new_pos[0] -= 1
10    elif action==1 and new_pos[0]<n-1: #move down
11        new_pos[0] += 1
12    elif action==2 and new_pos[1]>0: #move left
13        new_pos[1] -= 1
14    elif action==3 and new_pos[1]<n-1: #move right
15        new_pos[1] += 1
16    new_pos = tuple(new_pos)
17
18    current_reward = reward[current_pos]
19
20    if new_pos not in terminals:
21        #此处实现Q-learning算法
22        # 未到达最终状态,则采用贝尔曼方程更新Q函数
23        Q[current_pos][action] += alpha * (
24            current_reward + gamma * max(Q[new_pos]) - Q[current_pos][action]
25        )
26
27    else:
28        # 此处实现Q-learning算法
29        # 到达最终状态,则直接用当前奖励更新Q函数
30        Q[current_pos][action] += alpha * (current_reward - Q[current_pos][action])
31
32
33
34    if epsilon > 0.05:
35        epsilon -= epsilon_delta
36    current_pos = new_pos
```

Figure 5: Q-learning 算法补全

任务 2：运行并完成迷宫导航任务

提交运行结果截图，并进行算法分析。

下面是 `q_learning.py` 运行窗口初始状态和最终状态的截图。

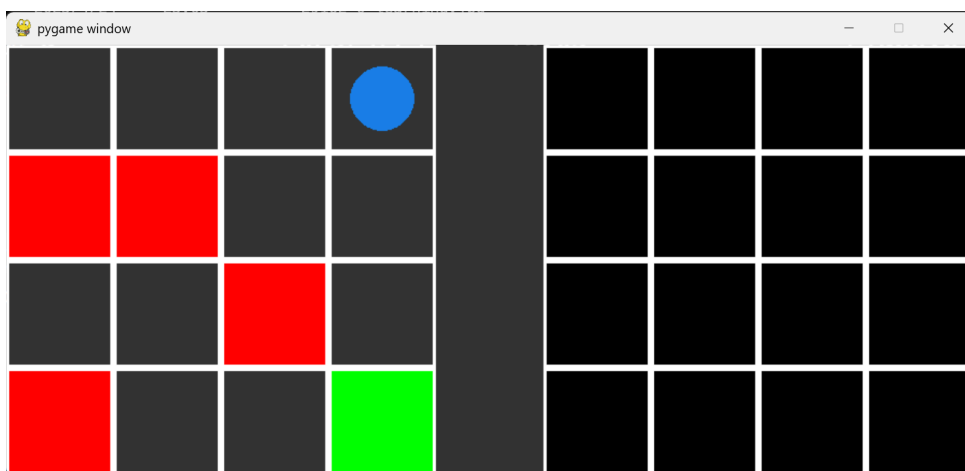


Figure 6: 迷宫导航任务初始状态

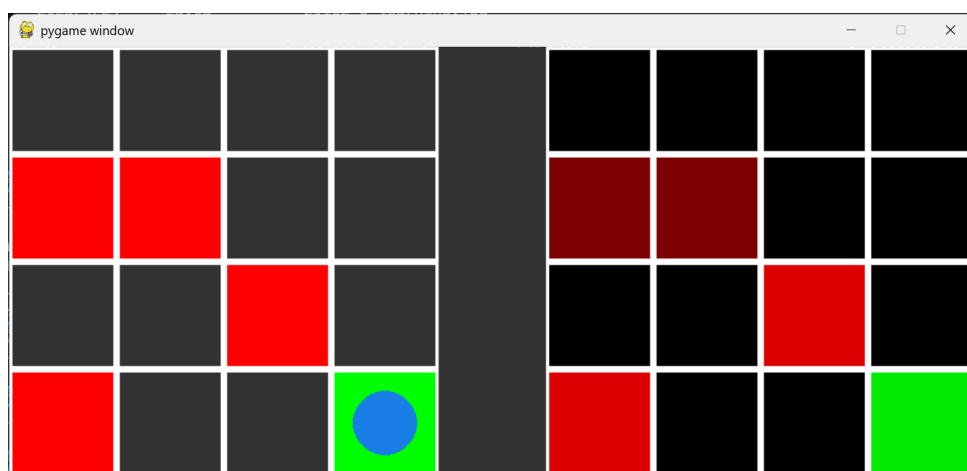


Figure 7: 迷宫导航任务中间状态

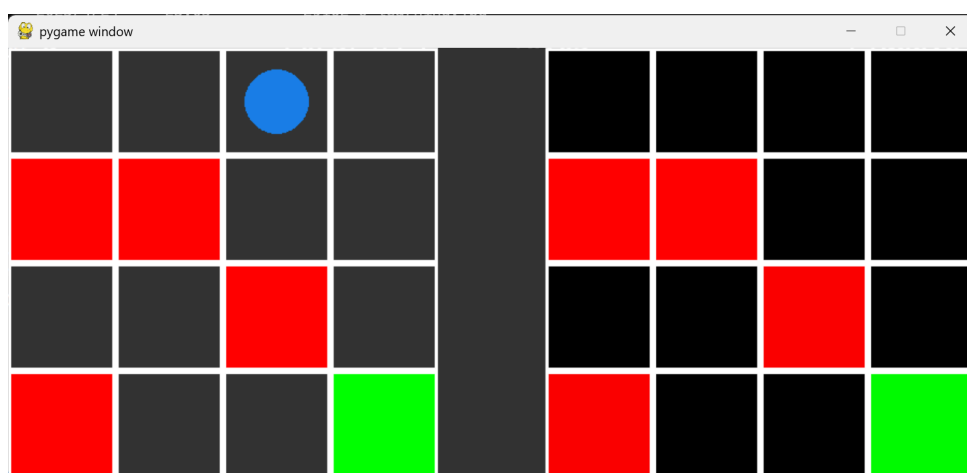


Figure 8: 迷宫导航任务最终状态

可以看到，初始状态下，Agent 对于迷宫中的障碍物和目的地并没有认知，右侧 V 值的可视化界面为全黑。随着 Agent 的不断探索，Agent 逐渐对迷宫中的障碍物和目的地有了认知，右侧 V 值的可视化界面逐渐变为的障碍物部分的红色色调逐渐变亮，目的地部分的绿色色调逐渐变亮。最终，Agent 成功找到了从起点到终点的路径，相应的障碍物和目的地也被用正确的颜色标出。

第三部分 利用 DQN 模型玩 Flappy bird 游戏以及 DQN 算法的改进

注：为了方便起见，本部分除了作业要求外，对工程代码做了一些其他修改。所有的修改附近我都添加了以“修改：”开头的注释。其中的部分修改，我也会在下面提到。

任务 1：测试训练 DQN 模型并进行演示

理解所给 Project 的逻辑，训练模型，利用模型玩 Flappy bird 游戏，并提交演示视频。

Pytorch 版本

训练模型，运行以下命令：

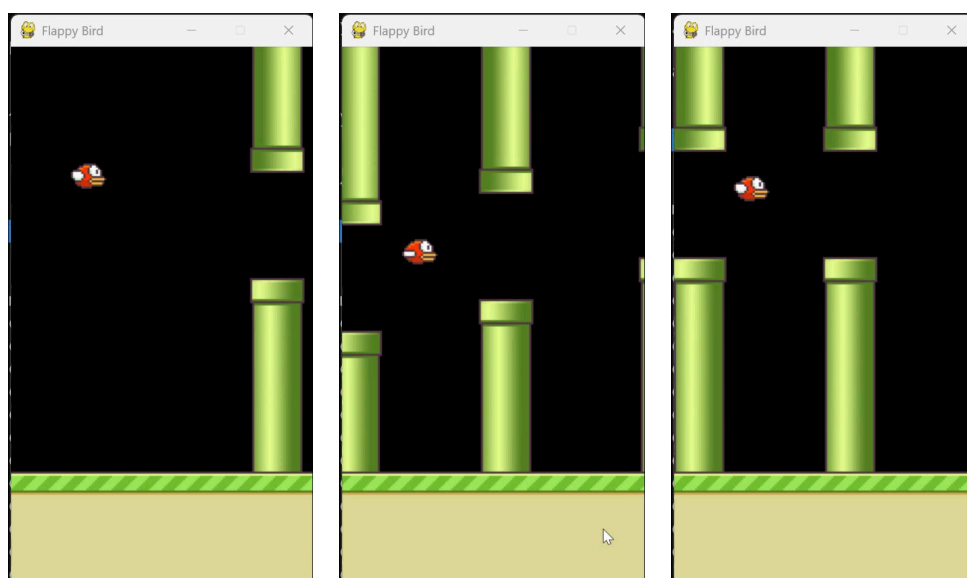
```
python dqn.py train
```

每隔 25000 个 `iteration` 就会保存一次模型。代码中预先设定的最高训练次数 `number_of_iterations` 为 2000000。

训练完成后，准备进行演示。为了方便选择模型，我在代码中添加了通过 `iteration` 选择模型的逻辑。运行以下命令以进行模型测试，其中 `<iteration>` 为你想要选择的模型的 `iteration`。默认值为 2000000。：

```
python dqn.py test <iteration>
```

运行测试，根据测试结果，统计得到不同迭代次数下小鸟能顺利通过管道的次数：



30 万 1-2 次

80 万 12-18 次

200 万 几乎死不掉

可以看到，随着模型训练迭代次数的增加，模型的表现也越来越好。

Tensorflow 版本

对于 TensorFlow 版本的工程，特别要提到的一点是，原本的代码中虽然能够在训练（同时也在展示）过程中“加载”模型文件，但是从实现效果可以看出，这一“加载”操作无论对于在中途中断训练后从断点开始训练，还是对于展示出的加载断点处保存的模型，均没有起到作用。

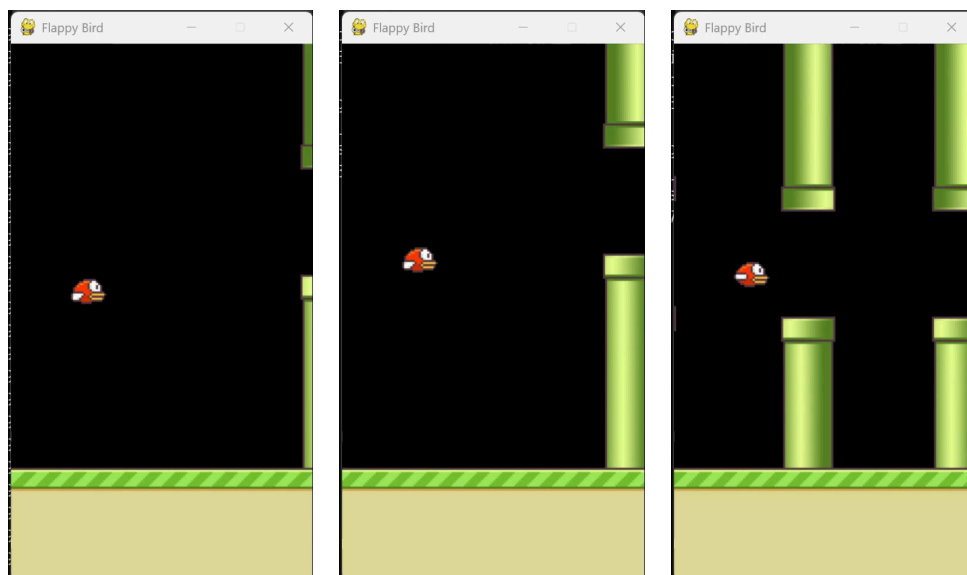
我在训练时租用了 AutoDL 云服务器，但是在云服务器上难以显示游戏的图形窗口，因此需要在服务器端训练，下载模型文件到本地之后，接着运行相关文件继续训练并演示。而这个问题的存在影响到了我的这一算力使用策略。

通过分析代码实现逻辑，发现原因在于原来的工程中只对于网络参数进行了保存，而并没有保存 `timeStep`、`epsilon`、`replayMemory` 等非网络参数，但是对于训练与演示过程也有影响的（比如 `epsilon` 的值直接影响着动作选择的策略）参数。因此，我在原有的代码基础上，添加了对这些参数的保存和加载逻辑。

训练并演示模型，运行以下命令：

```
python FlappyBirdDQN.py
```

根据演示结果，统计得到不同迭代次数下小鸟能顺利通过管道的次数：



25 万 1-2 次

70 万 6-10 次

160 万 几乎死不掉

可以看到，随着模型训练迭代次数的增加，模型的表现也越来越好。

任务 2：为 DQN 模型添加 Target Network 机制

为 DQN 模型添加 Target Network 机制，提交 Target Network 实现的代码截图，并描述其实现思路。

Pytorch 版本

下面是 `dqn_addTargetNetwork.py` 中添加 Target Network 机制的代码实现截图。

```
1 # 修改: 添加对Target Network的初始化
2 # initialize Target Network
3 target_model = NeuralNetwork()
4 if torch.cuda.is_available(): # put on GPU if CUDA is available
5     target_model = target_model.cuda()
6 target_model.load_state_dict(model.state_dict()) # Copy weights from the main model
7 target_model.eval() # Set Target Network to evaluation mode
```

Figure 9: Target Network 初始化

初始化部分, 创建一个 `NeuralNetwork` 实例, 将其赋值给 `target_model` 变量, 并相应的使用 GPU 加速。之后, 将当前模型的参数复制到 `target_model` 中。

使用 `eval()` 方法将 `target_model` 设置为评估模式, 目的是为了让它在推理时稳定输出, 不引入训练态的随机性和参数漂移。

```
1 # 修改: 使用 Target Network 的输出计算目标 Q 值 (y_batch), 而不是直接使用主网络的输出。
2 # get output for the next state from Target Network
3 output_1_batch = target_model(state_1_batch)
```

Figure 10: Target Network 计算 Q 值

使用 Target Network 的输出来计算 Q 值, 而不是直接使用主网络的输出。

```
1 # 修改: 添加对Target Network的更新
2 # Update Target Network every 10,000 iterations
3 if iteration % 10000 == 0:
4     target_model.load_state_dict(model.state_dict())
5     print("Updated Target Network")
```

Figure 11: Target Network 更新

每隔 10000 次迭代, 将当前模型的参数复制到 `target_model` 中, 对其进行更新。

Tensorflow 版本

下面是 `BrainDQN_NIPS_addTargetNetwork.py` 中添加 Target Network 机制的代码实现截图。

```
1 TARGET_UPDATE_FREQUENCY = 1000 # 修改: 每隔1000次更新Target Network
2
3 class BrainDQN:
4
5     def __init__(self, actions):
6         # 修改: 将replayMemory保存在pickle文件当中, 以便后续重新加载
7         # init replay memory
8         self.replayMemory = deque()
9
10        # init some parameters
11        self.timeStep = 0
12        self.epsilon = INITIAL_EPSILON
13        self.actions = actions
14        # init Q network
15        self.createQNetwork()
16        # 修改: 添加Target Network的创建
17        self.createTargetNetwork()
```

Figure 12: 类的初始化部分的更改

这一部分添加了 `TARGET_UPDATE_FREQUENCY` 变量, 表示 Target Network 更新的频率, 每隔 1000 个 `timeStep` 更新一次。另外, 还在 `__init__` 函数中添加了对 Target Network 的初始化逻辑。

```
1 # 修改: 添加了Target Network的创建
2 def createTargetNetwork(self):
3     self.targetQValue = tf.identity(self.QValue)
4     self.updateTargetNetwork = []
5     q_vars = tf.trainable_variables()
6     for var in q_vars:
7         self.updateTargetNetwork.append(var.assign(var))
```

Figure 13: Target Network 初始化

初始化逻辑的实现。与 Pytorch 版本过程类似, 先通过 `identity()` 函数创建一个与当前模型相同的模型, 然后将其赋值给 `target_model` 变量。之后, 利用 `assign()` 函数将当前模型的参数复制到 `target_model` 中。

其中还添加了一个列表 `updateTargetNetwork`, 用于存储更新 Target Network 权重的操作。这些操作会在训练过程中定期执行, 将主网络的权重复制到 Target Network。

```

1 # 修改: 使用Target Network来计算Q值
2 QValue_batch = self.targetQValue.eval(feed_dict={self.stateInput: nextState_batch})
3 # QValue_batch = self.QValue.eval(feed_dict={self.stateInput:nextState_batch})
4 for i in range(0,BATCH_SIZE):
5     terminal = minibatch[i][4]
6     if terminal:
7         y_batch.append(reward_batch[i])
8     else:
9         y_batch.append(reward_batch[i] + GAMMA * np.max(QValue_batch[i]))

```

Figure 14: Target Network 计算 Q 值

这一部分添加了 `target_model` 的计算逻辑。与 Pytorch 版本过程类似, 使用 Target Network 的输出来计算 Q 值, 而不是直接使用主网络的输出。

```

1 # 修改: 每隔1000次更新Target Network
2 if self.timeStep % TARGET_UPDATE_FREQUENCY == 0:
3     self.session.run(self.updateTargetNetwork)
4     print("Updated Target Network")

```

Figure 15: Target Network 更新

当 `timeStep` 整除 `TARGET_UPDATE_FREQUENCY` 时, 通过 `session.run()` 函数执行 `updateTargetNetwork` 列表中的操作, 将当前模型的参数复制到 `target_model` 中。

任务 3: 训练改进后的 DQN 模型、演示以及与改进前的对比分析

重新训练模型, 利用模型玩 Flappy bird 游戏, 提交演示视频; 绘制训练过程中奖励的变化图(包括两条曲线: 原始 DQN 和加了 Target Network 的 DQN), 并进行算法分析(Target Network 对模型的影响)。

Pytorch 版本

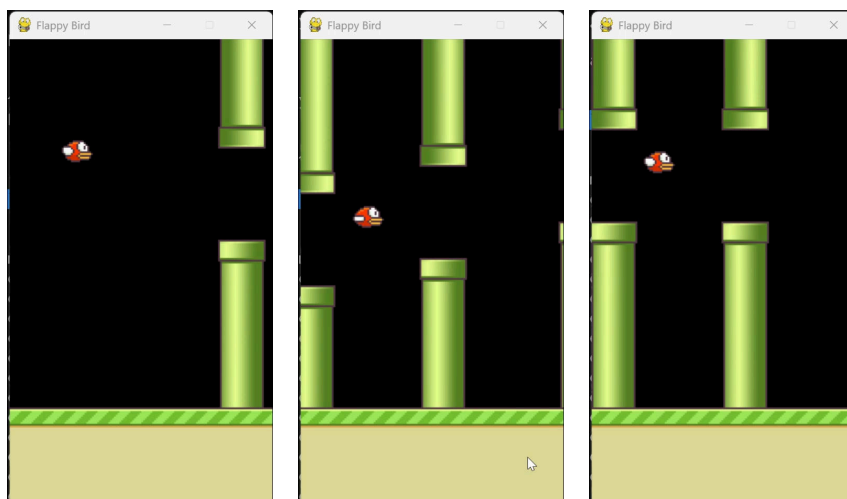
重新训练模型, 利用得到的模型进行演示。

```

python dqn_addTargetNetwork.py train
python dqn_addTargetNetwork.py test <iteration> # 默认值为2000000

```

得到不同迭代次数下小鸟能顺利通过管道的次数:



30 万 1-2 次

80 万 16-20 次

200 万 几乎死不掉

可以看出，在训练中期，模型的表现有了明显的提升。由于 Pytorch 版本的工程中没有给出随着 episode 变化而变化的 reward 的记录，因此这里就不展示 reward 的变化图了。

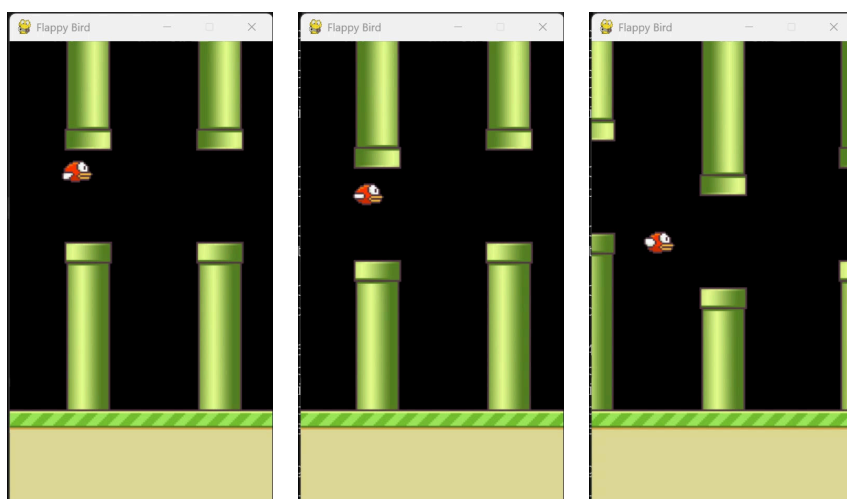
Tensorflow 版本

重新训练模型，利用得到的模型进行演示。为了切换运行有 Target Network 的模型和没有 Target Network 的模型，我在代码中添加了一个参数，运行以下命令：

```
python FlappyBirdDQN.py <useTargetNetwork>
```

其中 `<useTargetNetwork>` 为 `False`（默认值）或 `True`，分别表示不使用 Target Network 和使用 Target Network。

得到不同迭代次数下小鸟能顺利通过管道的次数：



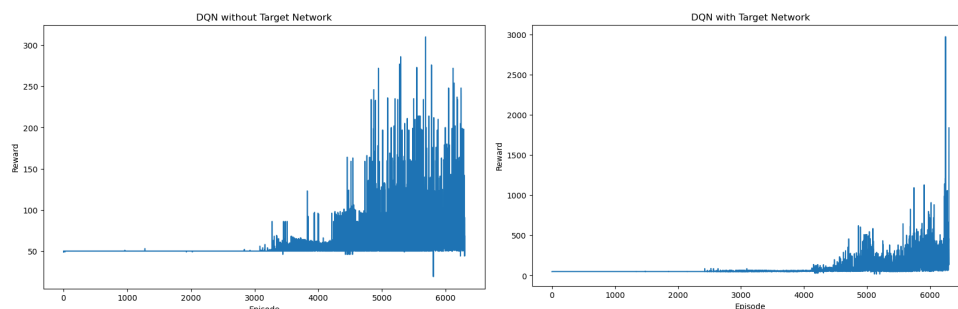
20 万 1-5 次

55 万 6-10 次

135 万 几乎死不掉

可以看出，达到与不加 Target Network 的模型相同效果所需要的迭代次数明显减少。

TensorFlow 版本的工程提供了对于 reward 的记录，因此可以绘制出 reward 随 episode 的变化图。



reward 随 episode 的变化图

reward 随 episode 的变化图（添加 Target Network）

注意两张图的 reward 的纵坐标尺度，右图的纵坐标尺度是左图的 10 倍。

为了更清晰的对比，可以将两张图合并到一起：

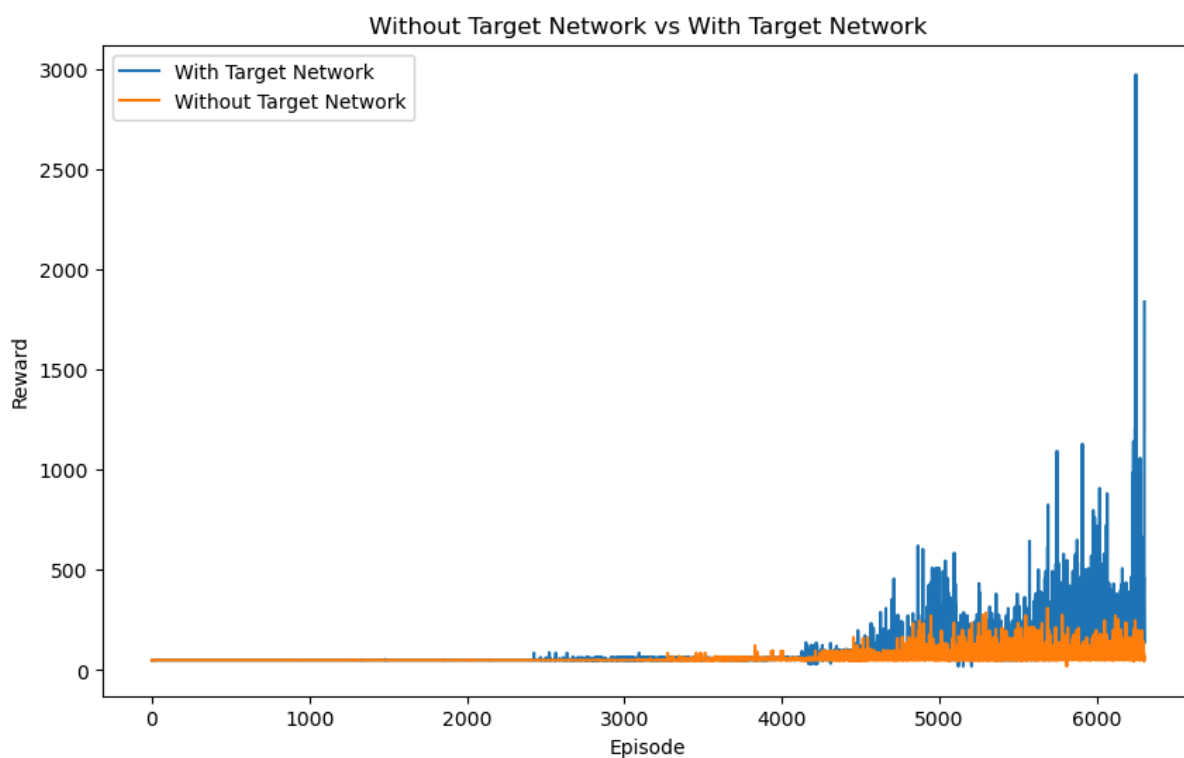


Figure 16: reward 随 episode 的变化图

可以看到，添加 Target Network 后，模型的收敛速度明显加快，最终的表现也更加稳定。

Target Network 对模型的影响

从 reward 随 episode 的变化图可以看出, 添加 Target Network 后, 模型的收敛速度明显加快, 最终的表现也更加稳定。这是因为 Target Network 的引入有效缓解了训练过程中 Q 值更新的不稳定性。

在原始 DQN 中, Q 值的更新依赖于当前网络的预测值, 而当前网络的参数在训练过程中不断变化, 这可能导致目标值 (target) 和预测值之间的关系不稳定, 从而影响训练效果。而 Target Network 通过引入一个独立的网络来计算目标值, 并定期将当前网络的参数复制到 Target Network 中, 避免了目标值和预测值之间的直接耦合, 从而提高了训练的稳定性 and 收敛速度。

因此, Target Network 是 DQN 算法中一个重要的改进机制, 能够显著提升模型的训练效果和稳定性。

附录：提交文件清单及说明

报告文档

- 机器学习2025作业3_陈墨霏_1120233329.docx : 作业报告 Word 文档

第一部分

- RL.py : Q-learning 算法的实现代码
- TestRL.py : Q-learning 算法的简单测试代码
- TestMaze.py : Q-learning 算法的复杂测试代码

第二部分

- q_learning.py : 迷宫导航任务的代码

第三部分

- Flappybird-pytorch/ : Pytorch 版本的 DQN 玩 Flappy bird 工程代码。由于模型文件过大，因此仅包含最后一个模型文件。
- Flappybird-tensorflow/ : Tensorflow 版本的 DQN 玩 Flappy bird 工程代码。此外，其中还包含了 draw.ipynb，用于绘制 reward 随 episode 的变化图。由于模型文件过大，因此仅包含最后一个模型文件（其中还包含了我为了使得工程能够正常的从训练与演示断点正常加载模型而包含的其他相关训练时产生的模型文件）。
- 演示视频/ : 演示视频文件夹，包含了 Pytorch 版本和 Tensorflow 版本的最好训练模型的演示视频。
 - tf_tnn_1350000.mp4 : TensorFlow 版本，添加 Target Network, timeStep=1350000。
 - tf_without_tnn_1600000.mp4 : TensorFlow 版本，不添加 Target Network, timeStep=1600000。
 - torch_tnn_2000000.mp4 : Pytorch 版本，添加 Target Network, iteration=2000000。
 - torch_without_tnn_2000000.mp4 : Pytorch 版本，不添加 Target Network, iteration=2000000。